Home Assignment: CAM Reconciliation Intelligence Prototype

REAL is building a **system of record and action for commercial real estate**, enabling operators to:

- Understand complex real estate data (leases, financials, documents)
- Surface insights across portfolios
- Execute workflows through intelligent agents

The platform combines:

- Structured + unstructured data
- AI-driven extraction and reasoning
- Action-oriented workflows for real estate teams

This assignment reflects a **real class of problems** we solve.

Context

Commercial tenants are billed annually for **Common Area Maintenance (CAM)** expenses by landlords. These reconciliations are often:

- Opaque and hard to validate
- Spread across multiple documents (leases, invoices, statements)
- Error-prone, sometimes materially so
- Reviewed manually with significant time investment

As a result, tenants and operators struggle to answer a simple question:

> **"Are we overpaying, and if so—where and why?"**

Your Mission

Design and prototype a **CAM Reconciliation Intelligence experience** that helps a user **quickly understand, validate, and act on a CAM reconciliation**.

This is **not** a design exercise or a PRD.
This is a **product thinking + execution** challenge.

What to Build

Create a **working prototype** (using any tools you prefer—AI tools, no-code, code, Figma + logic, etc.) that demonstrates:

- How a user interacts with a CAM reconciliation
- How insights are surfaced
- How decisions or actions are enabled

You are free to define:

- The target user (asset manager, finance, operator, etc.)
- The scope (single location vs portfolio)
- The level of automation vs assistive intelligence
- The data inputs and structure

Guiding Prompt (Do Not Overfit To This)

A user receives:

- A CAM reconciliation statement (annual summary)
- A lease document
- Supporting cost breakdowns / invoices (optional)

They want to:

- Validate charges
- Identify discrepancies
- Understand financial impact
- Decide what to do next

What We're Looking For

We are intentionally **not prescribing structure**.
Your submission should make clear:

1. Problem Framing

- What problem did you *choose* to solve (not all of them)?
- What assumptions did you make?
- What did you intentionally ignore?

2. Product Thinking

- What is the **core user flow**?
- Where is the **moment of value**?
- What makes this better than current workflows?

3. Experience

- How does the user go from:
    - Raw data → insight → action
- How are **confidence, trust, and explainability** handled?

4. Prototype

- A working prototype or interactive flow
- Doesn't need to be production-ready, but should be **convincing**

5. Tradeoffs

- What did you simplify?
- Where would this break in real-world usage?
- What would you build next?

Constraints (Light, On Purpose)

- You may use **any tools** (LLMs, no-code, scripts, UI tools, etc.)
- You may **mock or synthesize data**
- You may define your own schema and structure
- You do **not** need full backend logic—but the experience should feel real

What Strong Submissions Usually Show

(Not requirements—just directional signals)

- A clear opinion on **what matters vs noise**
- Ability to reduce a complex domain into **actionable clarity**
- Thoughtful handling of **ambiguity and incomplete data**
- A strong sense of **“so what?” for the user**
- Evidence of **product judgment**, not just building

Submission

Please provide:

- A short walkthrough (Loom or similar)
- Access to your prototype
- Any supporting notes (optional)

Time Expectation

We expect candidates to spend **~2-3 hours**.
Depth > breadth.

Final Note

There is no “correct” solution.

We care about:

- How you think
- Where you focus
- What you choose *not* to build